

Atividade Prática – Teste de API REST utilizando Postman

Objetivo

Realizar testes funcionais em uma API REST desenvolvida em PHP, validando o comportamento dos métodos HTTP, parâmetros enviados na URL, dados enviados no corpo da requisição (Body) e códigos de resposta retornados pelo servidor.

Cenário

Uma equipe de desenvolvimento criou uma API para gerenciamento de usuários.

Sua função é atuar como Analista de Testes, verificando se a API responde corretamente para situações válidas e inválidas.

Todos os testes deverão ser realizados utilizando o Postman.

Parte 1 – Conhecendo a API

Antes de iniciar os testes, responda:

1. Quais métodos HTTP são suportados pela API?

GET - CONSULTAR/BUSCAR DADOS EXISTENTES

POST - CRIAR/ENVIAR NOVOS REGISTROS

PUT - SUBSTITUIR/ATUALIZAR RECURSO COMPLETO

PATCH - ATUALIZAR PARCIALMENTE O REGISTRO

DELETE - REMOVER UM REGISTRO ESPECIFICADO

2. Quais métodos retornam erro caso sejam utilizados incorretamente?

TODOS

3. Quais métodos exigem:

ID na URL

GET

PUT

PATCH

DELETE

Dados no Body

POST

Ambos

PUT

PATCH

Parte 2 – Testes do Método GET

Caso de Teste 01

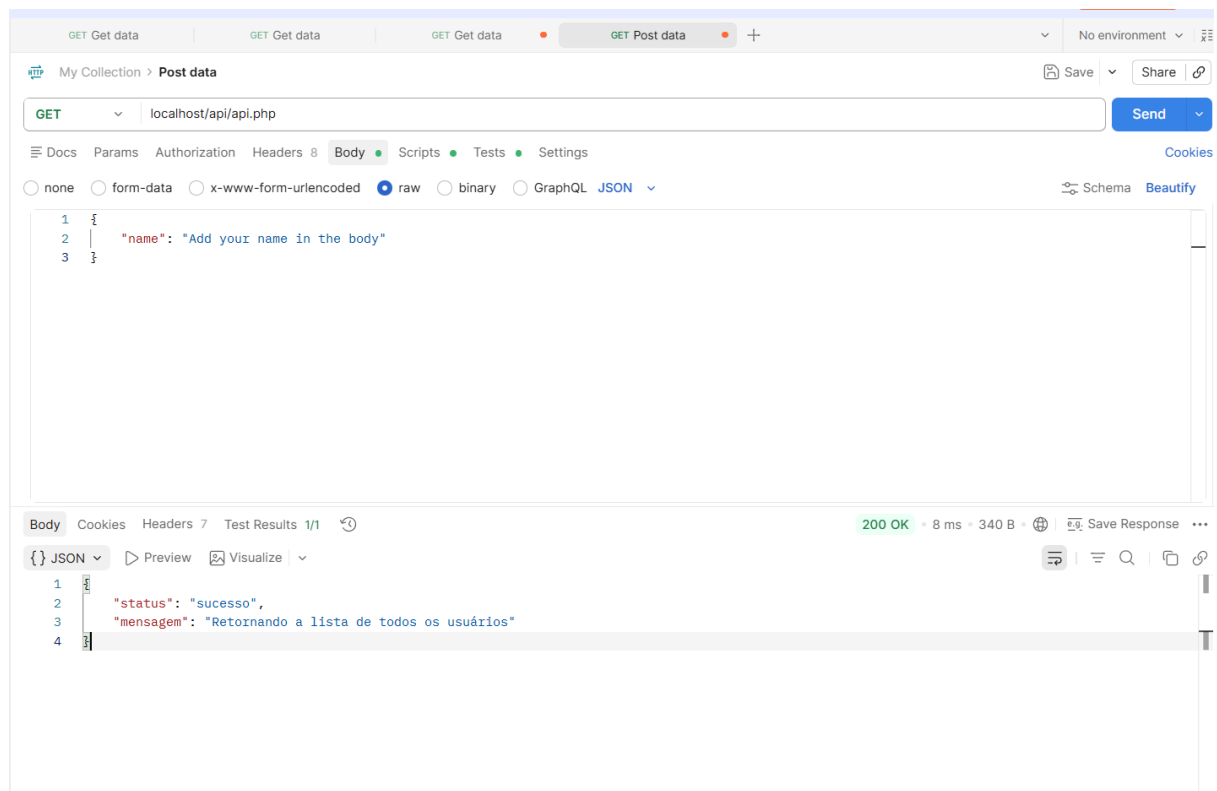
Consultar todos os usuários.

Registrar:

URL utilizada:

Código HTTP retornado:

Corpo da resposta:



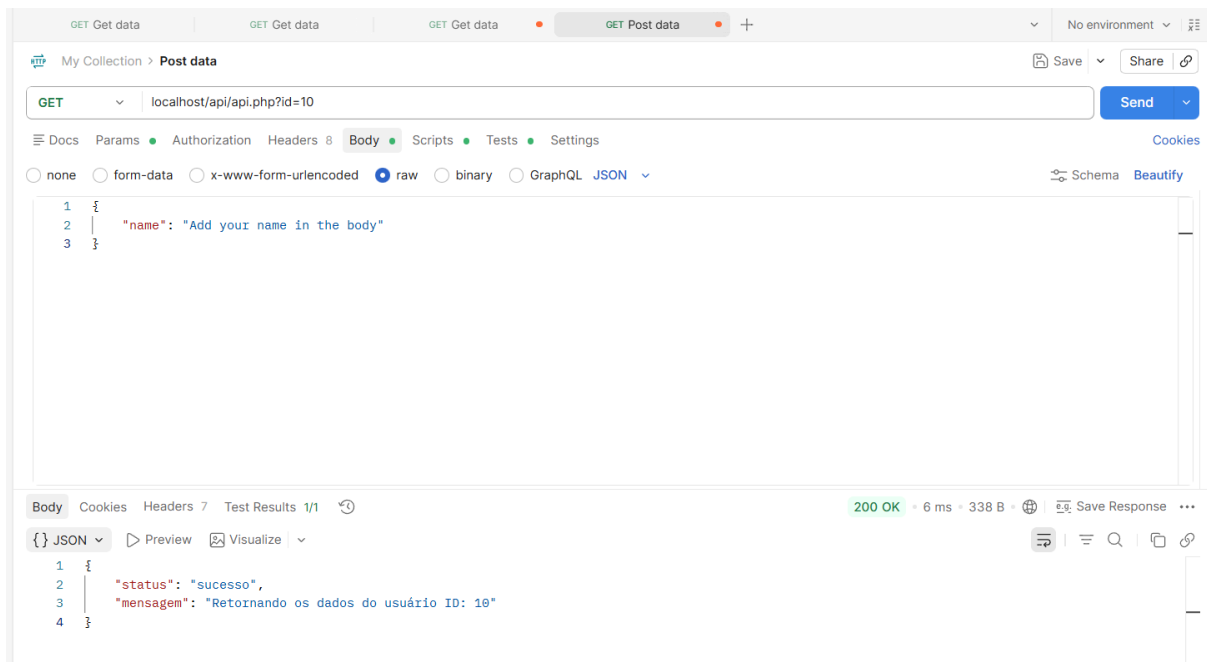
Caso de Teste 02

Consultar um usuário específico informando um ID.

Registrar:

- URL utilizada:
- Código HTTP retornado:

- Corpo da resposta:



Parte 3 – Testes do Método POST

Caso de Teste 03

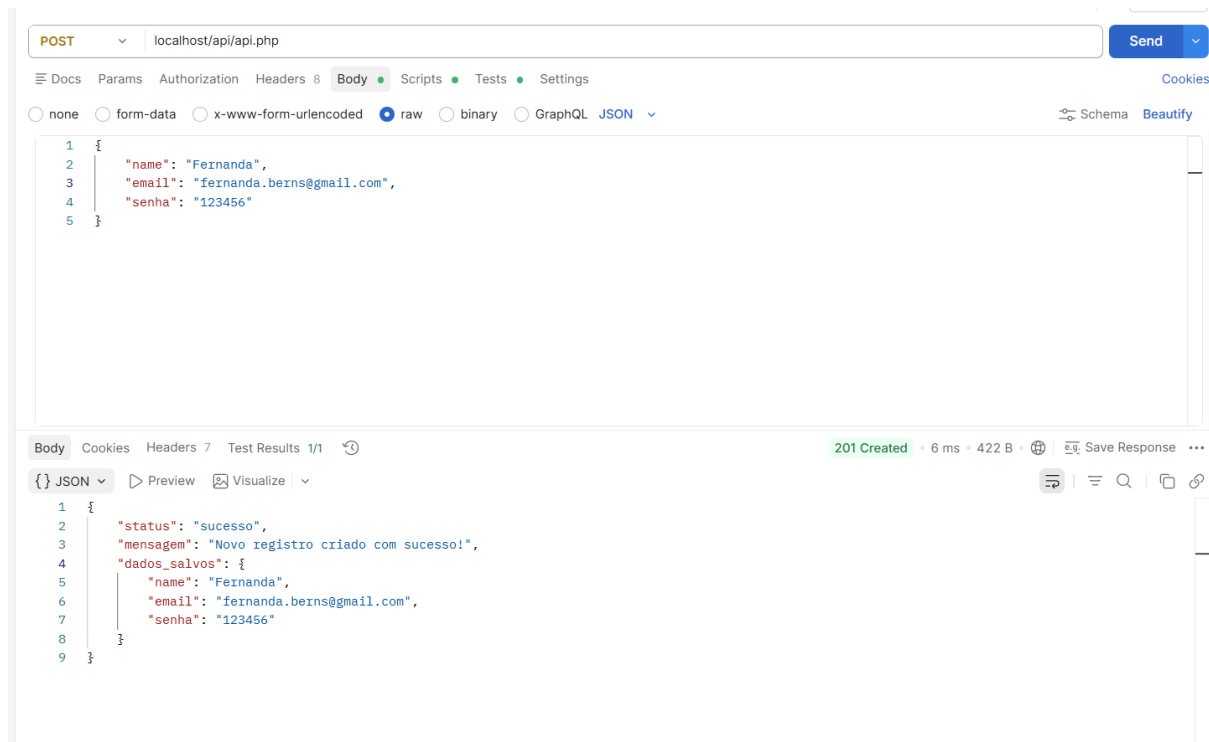
Cadastrar um novo usuário enviando:

- Nome
- Email
- Senha

Utilize JSON no Body.

Registrar:

- JSON enviado:
- Código HTTP retornado:
- Resposta da API:

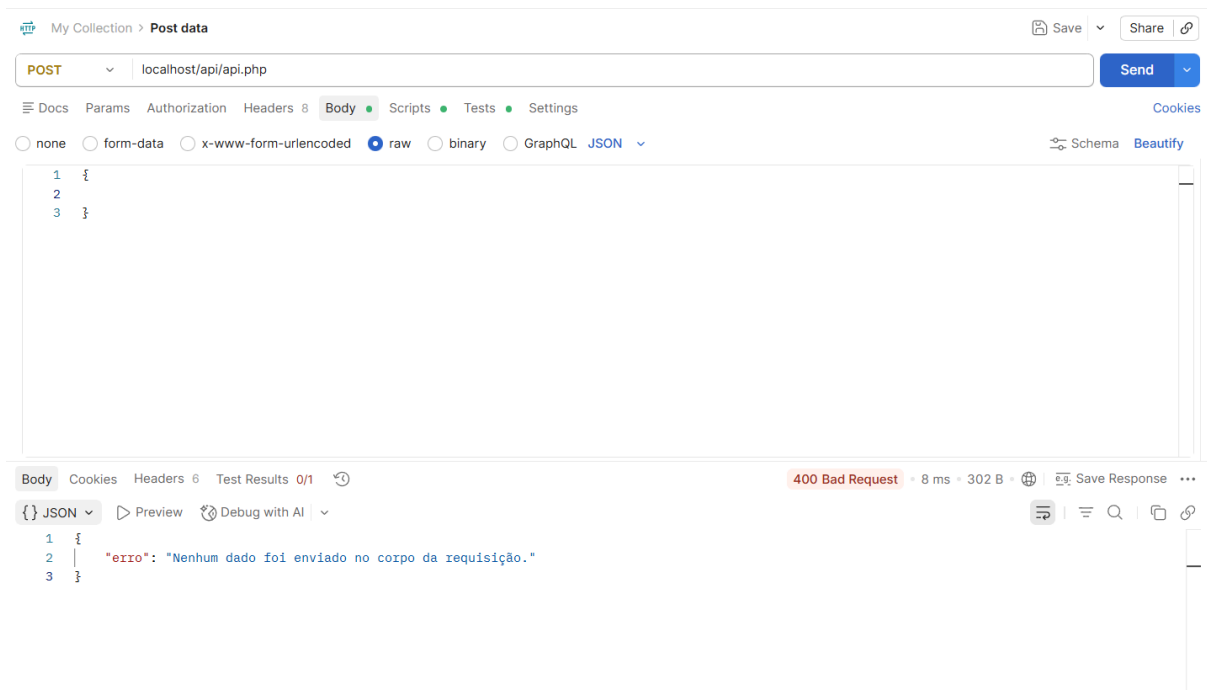


Caso de Teste 04

Executar um POST sem enviar Body.

Responder:

- Qual código HTTP foi retornado?
- Qual mensagem foi exibida?



Parte 4 – Testes do Método PUT

Caso de Teste 05

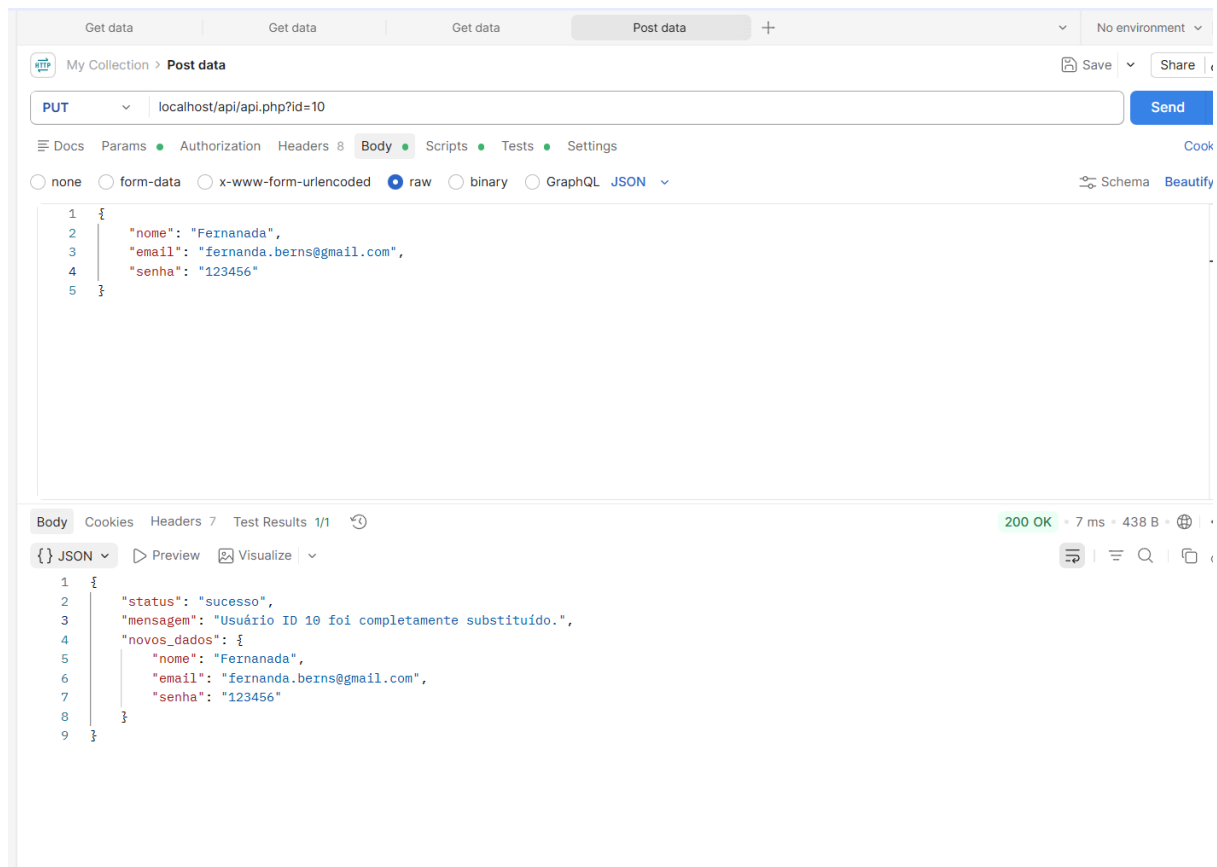
Atualizar completamente um usuário existente.

Enviar:

- ID na URL
- Dados completos no Body

Registrar:

- URL utilizada:
- JSON enviado:
- Código HTTP retornado:
- Resposta da API:

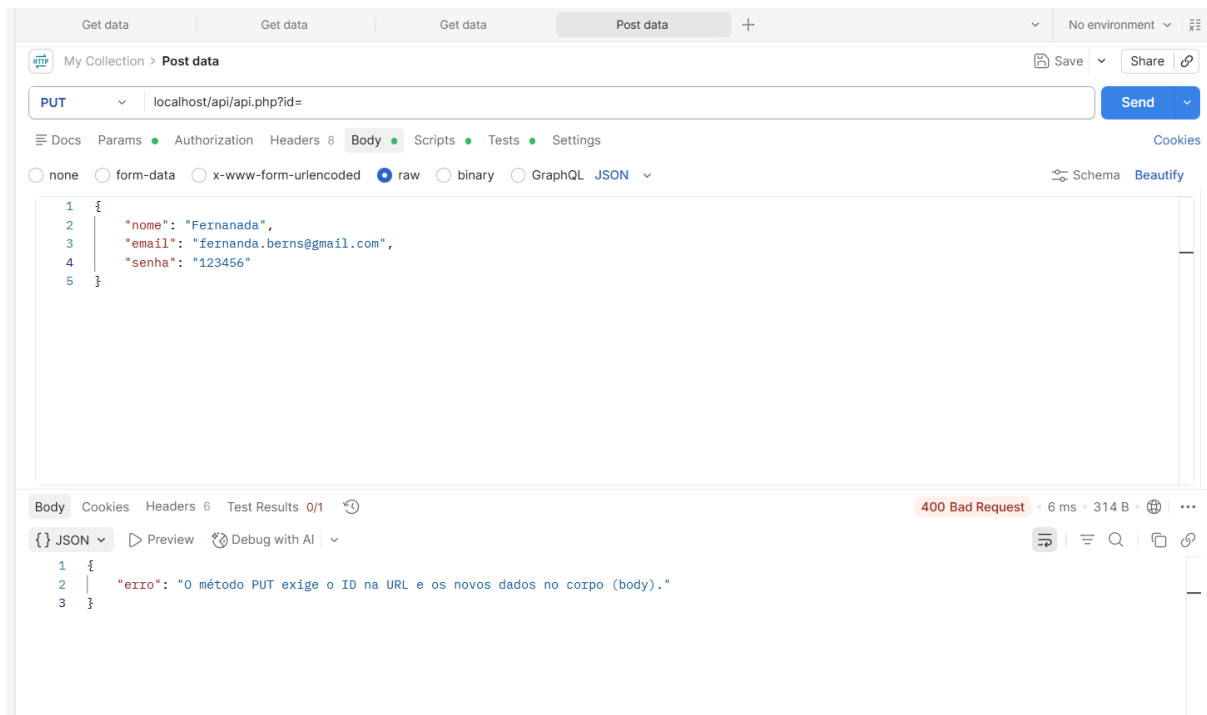


Caso de Teste 06

Executar um PUT sem informar o ID.

Responder:

- O sistema permitiu a operação?
- Qual código HTTP foi retornado?
- Qual mensagem foi exibida?

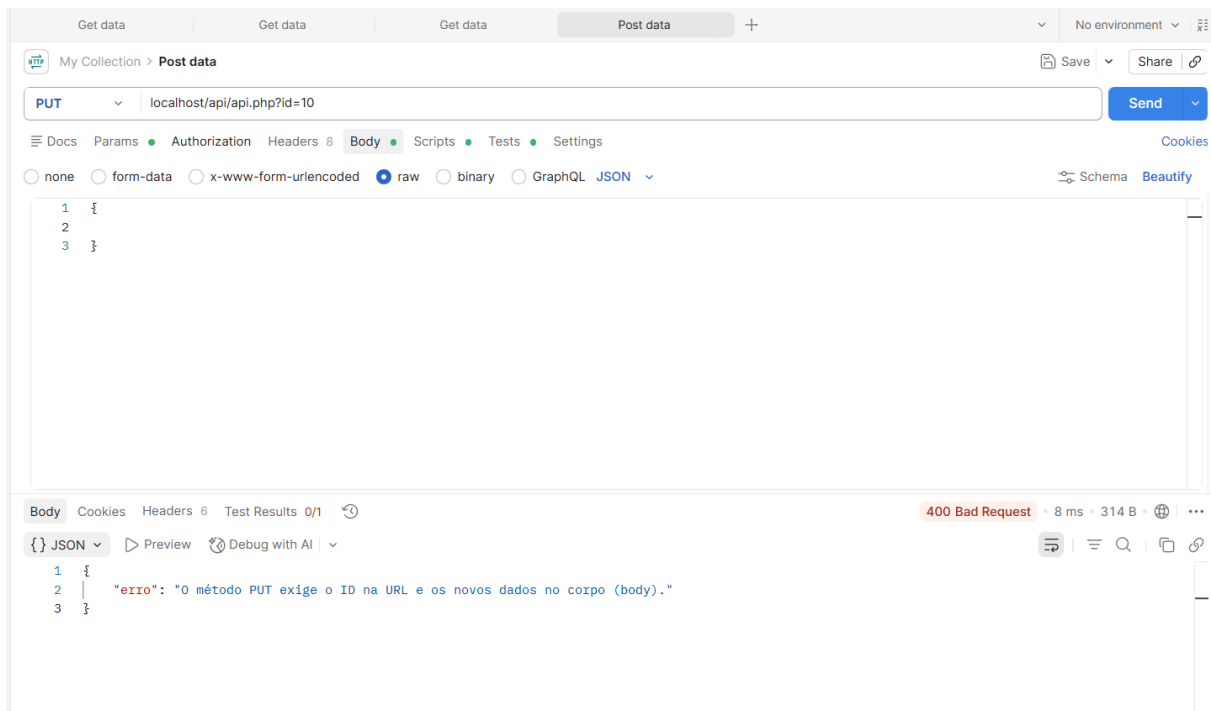


Caso de Teste 07

Executar um PUT sem enviar dados no Body.

Responder:

- Qual código HTTP foi retornado?
- Qual mensagem foi exibida?



Parte 5 – Testes do Método PATCH

Caso de Teste 08

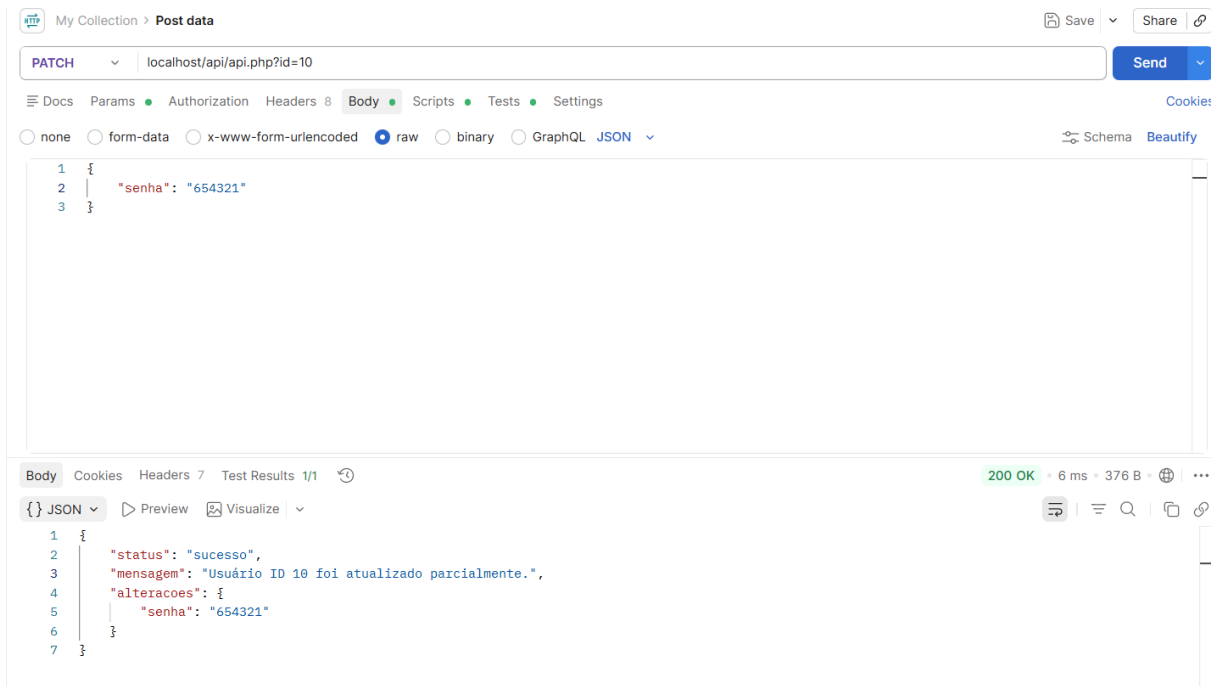
Alterar apenas um campo do usuário.

Exemplo:

- Senha

Registrar:

- URL utilizada:
- JSON enviado:
- Código HTTP retornado:
- Resposta da API:

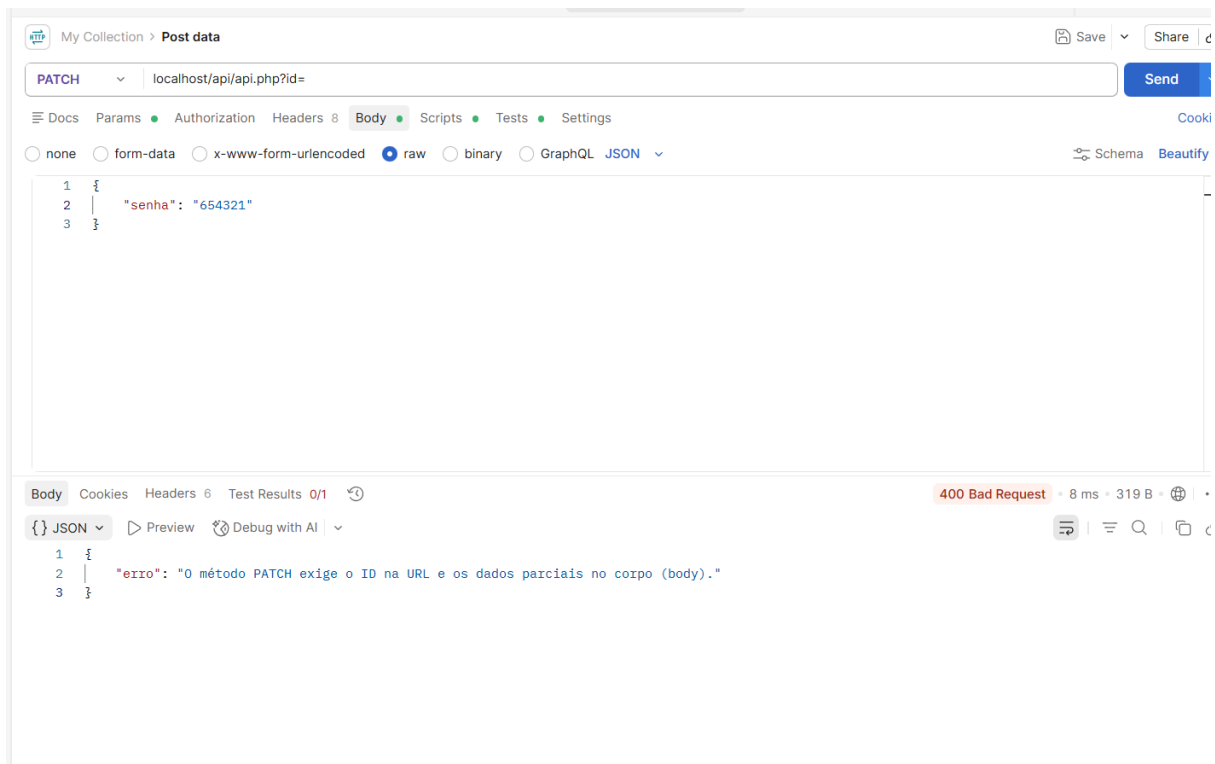


Caso de Teste 09

Executar um PATCH sem informar o ID.

Responder:

- O sistema permitiu a operação?
- Qual código HTTP foi retornado?
- Qual mensagem foi exibida?



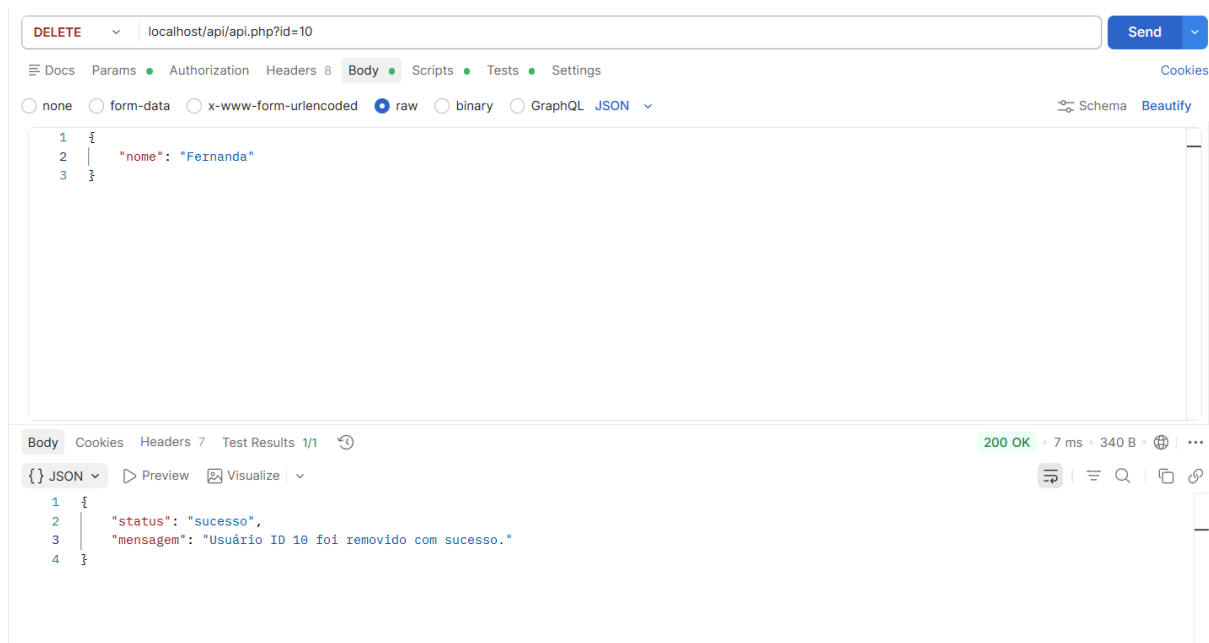
Parte 6 – Testes do Método DELETE

Caso de Teste 10

Excluir um usuário informando o ID.

Registrar:

- URL utilizada:
- Código HTTP retornado:
- Resposta da API:

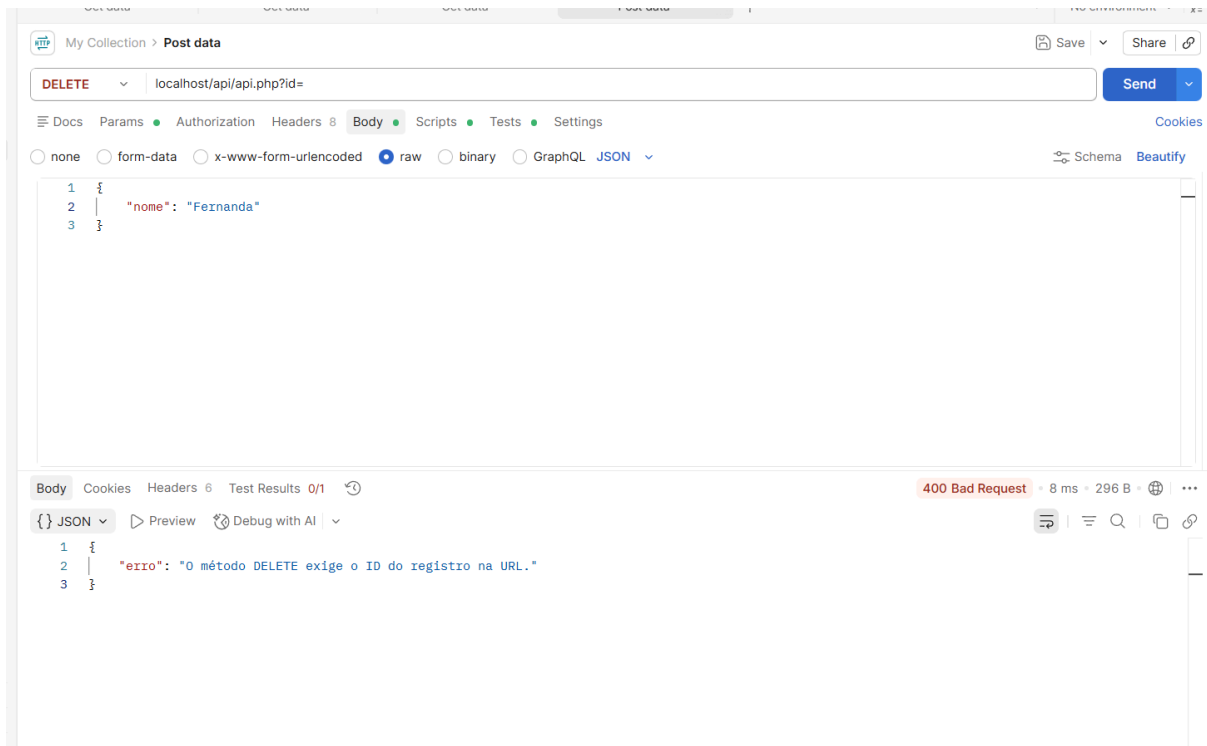


Caso de Teste 11

Executar um DELETE sem informar o ID.

Responder:

- O sistema permitiu a exclusão?
- Qual código HTTP foi retornado?
- Qual mensagem foi exibida?



Parte 7 – Testes de Erro

Caso de Teste 12

Enviar uma requisição utilizando um método HTTP não implementado pela API.

Exemplos:

- HEAD
- OPTIONS

Responder:

- Qual código HTTP foi retornado?
- Qual mensagem foi exibida?

Get data | Get data | Get data | **Post data** | + | No environment

My Collection > **Post data** | Save | Share

HEAD | localhost/api/api.php?id=10 | Send

Docs | Params | Authorization | Headers 8 | **Body** | Scripts | Tests | Settings | Cookie

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON | Schema Beautify

```
1 {
2   "nome": "Fernanda",
3   "email": "fernanda.berns@gmail.com",
4   "senha": "12346"
5 }
```

Body | Cookies | Headers 6 | Test Results 0/1 | 405 Method Not Allowed · 6 ms · 258 B | ...

{ } JSON | Preview | Debug with AI |

1

Get data | Get data | Get data | **Post data** | + | No environment

My Collection > **Post data** | Save | Share

OPTIONS | localhost/api/api.php?id=10 | Send

Docs | Params | Authorization | Headers 8 | **Body** | Scripts | Tests | Settings | Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON | Schema Beautify

```
1 {
2   "nome": "Fernanda",
3   "email": "fernanda.berns@gmail.com",
4   "senha": "12346"
5 }
```

Body | Cookies | Headers 7 | Test Results 0/1 | 405 Method Not Allowed · 8 ms · 370 B | ...

{ } JSON | Preview | Debug with AI |

```
1 {
2   "erro": "Método HTTP 'OPTIONS' não permitido ou não implementado nesta API."
3 }
```

Caso de Teste 13

Enviar um JSON inválido.

Exemplo:

```
{
```

```
  "nome": "João"
```

Responder:

- A API aceitou a requisição?
- O que aconteceu?
- Qual foi o código HTTP retornado?

The screenshot shows a REST client interface with a POST request to `localhost/api/api.php?id=10`. The request body is a JSON object: `{ "nome": "João" }`. The response is a `400 Bad Request` with a status of `7 ms` and a body of `302 B`. The response body is a JSON object: `{ "erro": "Nenhum dado foi enviado no corpo da requisição." }`.

My Collection > Post data

POST localhost/api/api.php?id=10

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "nome": "João"
```

Body Cookies Headers 6 Test Results 0/1

JSON Preview Debug with AI

```
1 {
2   "erro": "Nenhum dado foi enviado no corpo da requisição."
3 }
```

400 Bad Request 7 ms 302 B

Parte 8 – Análise dos Resultados

Após concluir todos os testes, responda:

1. Qual a diferença entre os métodos:

- GET - Buscar dados
- POST - Criar dados
- PUT - Atualizar tudo
- PATCH - Atualizar parte
- DELETE - Excluir dados

2. Qual a diferença entre PUT e PATCH?

PUT - Substitui o recurso inteiro com os novos dados.

PATCH - Altera somente os dados que foram informados.

3. Por que o método DELETE exige apenas o ID?

Por que ele precisa saber exatamente o que deletar

4. Por que o método POST retorna código HTTP 201 ao invés de 200?

Porque o 201 significa “recurso criado com sucesso”, enquanto o 200 significa apenas “requisição realizada com sucesso”.

5. Explique o significado dos códigos HTTP:

- 200 - requisição processada com sucesso
- 201 - [Created \(Criado com sucesso\)](#)
- 400 - [Bad Request \(Requisição inválida\)](#)
- 405 - [Method Not Allowed](#)

Relatório de Testes

Preencha a tabela abaixo para cada teste realizado.

Caso de Teste	Método HTTP	URL Utilizada	Código Retornado	Resultado Esperado	Resultado Obtido	Status
CT01	GET	localhost /api/api.php	200	listar todos os usuários	<pre>} "status": sucesso" "mensagem": "Retornando a lista com todos os usuários" }</pre>	sucesso
CT02	GET	localhost /api/api.php?id=10	200	listar o usuário com ID 10	<pre>} "status": sucesso" "mensagem": "Retornanddad os do usuário ID:10" }</pre>	sucesso
CT03	POST	localhost /api/api.php	201	criar dados	<pre>} "status": sucesso" "mensagem": "Novo registro criado com sucesso".</pre>	sucesso

					<pre> “dados_salvos” : { “name”: “Fernanda”, “email”: “fernanda.berns@hotmail.com”, “senha”: “123456” } } </pre>	
CT04	POST	localhost /api/api.php	400	ENVIAR DADOS	<pre> } “erro”: “Nenhum dado foi enviado no corpo da requisição.” } </pre>	erro
CT05	PUT	localhost /api/api.php?id=10	200	SUBSTITUIR DADOS	<pre> } “status”: sucesso” “mensagem”: “Usuario ID 10 foi completamente substituido”. “novos_dados” : { “name”: “Fernanda”, </pre>	sucesso

					<pre> “email”: “fernanda.bern s@hotmail.co m”, “senha”: “123456” } } </pre>	
CT06	PUT	localhost /api/api.p hp?id=	400	SUBSTIT UIR DADOS	<pre> { “erro”: O metodo PUT exige o ID na URL e os novos dados no corpo (body).” } </pre>	erro
CT07	PUT	localhost /api/api.p hp?id=10	400	SUBSTIT UIR DADOS	<pre> { “erro”: “O metodo PUT exige o ID na url e os novos dados do corpo (Body).” } </pre>	erro
CT08	PATCH	localhost /api/api.p hp?id=10	200	ATUALIZA R ALGUM DOS REQUISIT OS	<pre> } “status”: sucesso” “mensagem”: “Usuario ID 10 foi foi </pre>	sucesso

					atualizado parcialmente”. “alteracoes”: { “senha”: “654321” } }	
CT09	PATCH	localhost /api/api.php?id=	400	ATUALIZA R ALGUM DOS REQUISIT OS	{ “erro”: “o metodo PATCH exige o ID na URL e os dados parciais no corpo (body).” }	erro
CT10	DELET E	localhost /api/api.php?id=10	200	REMOVE R	} “status”: “sucesso”, “mensagem”: “Usuario ID 10 foi removido com sucesso.” }	sucesso
CT11	DELET E	localhost /api/api.php?id=	400	REMOVE R	{ “erro”: “O metodo DELETE exige o ID do registro na URL”	erro

					}	
CT12	OPTIONS	localhost /api/api.php?id=10	405		{ "erro": "Metodo HTTP 'Options' não permitido ou não implementado nessa API." }	erro
CT13	POST	localhost /api/api.php?id=10	400		{ "erro": "Nenhum dado foi enviado no corpo da requisição." }	erro

Entrega

O relatório deverá conter:

- Capturas de tela dos testes realizados no Postman.
- Método HTTP utilizado.
- URL utilizada.
- Código HTTP retornado.
- Resposta recebida da API.
- Respostas das questões teóricas.
- Conclusão final sobre o funcionamento da API.

Com os testes realizados no Postman, foi possível entender melhor como uma API funciona na prática. Cada método HTTP possui uma função específica e deve ser utilizado de maneira correta. Também foi possível verificar como a API responde a situações válidas e inválidas por meio dos códigos. Assim, os testes ajudaram a confirmar o funcionamento das operações de consulta, criação, atualização e exclusão de usuários, além de mostrar a importância do tratamento de erros na API.

Desafio Extra

Pesquise e explique:

1. O que é uma API REST.

É uma interface de programação de aplicações (API) que segue os princípios de design da arquitetura REST.

REST é a sigla para "Representational State Transfer" (transferência de estado representacional), um conjunto de regras e diretrizes sobre como você deve criar uma API web.

2. O que é JSON.

JavaScript Object Notation - é um formato de texto, usado para armazenar e transferir dados estruturados entre sistemas

3. Qual a função do Header `Content-Type`.

Informa ao navegador ou ao servidor qual é o tipo/formato do conteúdo que está sendo enviado ou recebido em uma requisição HTTP.

4. Qual a diferença entre `application/json` e `application/x-www-form-urlencoded`.

`Application/json` envia os dados no formato JSON, sendo muito utilizado em APIs.

`Application/x-www-form-urlencoded` envia os dados no formato de formulário, usando pares `chave=valor`, sendo comum em formulários HTML tradicionais.

5. Qual a função do Postman no processo de teste de sistemas.

É uma ferramenta utilizada para testar APIs, permitindo enviar requisições ao servidor e verificar as respostas, facilitando a identificação de erros e a validação do funcionamento do sistema.